

高速化チャレンジ restart v2 結果要約

2026年10月1日 10:25 JST 最終版

共通の公開APIで、SOM-OLPは12.90倍、T/C同時計算は最大8.33倍。k-meansは最終出力まで含めて低次元1.87倍 / 高次元1.20倍でした。

各課題・各条件の中だけで比較した代表候補の順位です。旧実験の数値は今回値に流用していません。

1 k-means

同じ初期中心、20反復、9スレッド。公開APIの最終割当・inertia計算まで包含 [1]

条件	順位	実装	時間	対sklearn
20,000 × 8 K=16	1	公開Numba + 直接最終割当	7.303 ms	1.87倍
	2	sklearn Lloyd	13.642 ms	基準
	3	自作NumPy参照 + 最終割当	134.323 ms	参照
10,000 × 128 K=64	1	公開Numba ベクトルBLAS + 最終割当	29.648 ms	1.20倍
	2	sklearn Lloyd	35.461 ms	基準
	3	自作NumPy参照 + 最終割当	3.219 s	参照

3方式とも最終割当・inertia込み。NumPy参照（特徴順の直接距離、B256）比は約18.4/108.6倍で、最速NumPy比ではありません。最終ラベルは完全一致、中心・inertiaは許容差内。時間には変動があり、先行窓2.12/1.21倍[6]と混ぜず最新窓を採用。直接最終割当は明示選択で、汎用sklearn互換の保証ではありません。

2 SOM-OLP

MNIST全70,000件 × 784次元、256ノード。初期化から収束36反復までの公開API [2]

順位	実装	時間	対元実装
1	公開ThreadPool + 低ランク初期確率式	13.627 s	12.90倍
2	元のSOMOLP.fit	175.724 s	基準

完全SVDを保ち、初期確率の低ランク式とThreadPoolを明示選択。入力検証・初期化・全反復・出力生成を計時内に含めました。初期化は両者9スレッド、最適化学習部は9ワーカー+BLAS 1スレッドです。別枠のnative 2.581秒は固定10反復・初期化除外のカーネル値で、上表と混ぜません。

3 Trustworthiness と Continuity の同時計算

公開APIの10,000件 × 64次元から2次元、float64、k=15、9スレッド [3]

順位	実装	時間	対sklearn 2回
1	公開 sqrt_numba 明示オプション	1.143 s	8.33倍
2	公開 numba scan	1.299 s	7.33倍
3	公開 numpy broadcast JIT不要	3.517 s	2.71倍
4	sklearn 2回	9.518 s	基準

全呼び出しでsklearnのスコアと参照整数ペナルティが一致。公開ラッパーを含む3回の交互ウォーム測定です。ブロック256行、32 MiBは順位計算の作業枠で、距離格納はO(N²)のままです。

測定条件 Linux x86_64 / AMD EPYC 9V74 / CPU 9スレッド / GPUなし。k-meansは7回、他は3回のウォーム中央値。初回は別記録、import・JIT準備・結果の数値検証は計時外です。単一環境・選定データの結果で、速度の一般保証ではありません。

OSと配布のしやすさ

共通Pythonパッケージを土台にし、必要な場所だけNumbaやThreadPoolを明示選択する構成です。速い設定が既定設定とは限らず、用途ごとの確認が必要です。

配布容易性	構成と導入負担	OSとCPUの位置づけ
1位 共通 標準依存版	NumPy / SciPy / sklearn等。独自C/C++コンパイラ・Cython不要。SOM ThreadPoolもNumba不要。	Linux x86_64は実測済み。Windows・macOS・ARMは移植候補で未検証。依存ライブラリの対応配布物が必要。
2位 共通 + Numba	追加依存はNumba / llvmlite。初回JIT時間が必要。対応wheel利用時は独自ビルド不要。	上流Tier 1はWindows x64、Linux x64 / ARM64、macOS Apple Silicon。この試作の実機確認はLinux x64のみ [7]。
3位 専用 Cython	GCC・OpenMP・libmvec等、CPU/OS依存のビルドが必要。標準パッケージの必須依存にはしない。	今回のLinux x86_64構成でのみ確認。Windows・macOS・ARMへのビルド移植・検証、汎用native wheelは未整備。

実機検証はすべてLinux x86_64のみです。上流ライブラリの対応と、この試作の動作確認は別です。Intel MacはNumbaのTier 1対象外で、0.63以降はTier 2bのコミュニティ対応です。NumPy経路を優先候補にします [7]。

正確さと検証範囲

SOM-OLPは $W \cdot P \cdot V$ ・全目的関数履歴が元実装と $rtol=atol=1e-8$ で一致し、反復数は同じ36回。各設定内の反復結果はビット単位で安定しましたが、元実装とのビット一致を意味しません。今回の全70k埋め込みについて、T/C整数ペナルティの再比較までは実施していません。

配布a2の独立インストール試験は14件中12成功・Numba専用2件スキップ。no-JITの3 APIとPreparedSOMの小規模一致も通過。配布33ソースを確認し、計時済み数値コア31ファイルは不変です。任意ランタイム込みの統合14件も別途通過。公開用全体ライセンスは未選定です [4]。

共通化と実用上の注意

別条件のPreparedSOMでは、同じ70k入力で γ/λ を変える3 fitが40.008→33.089秒（1.21倍、毎回の準備4.189秒込み）。出力・履歴はfreshと完全一致し、追加の所有配列は約420 MiB。1 fitでは遅く、この条件では2 fit以降に利益が出ました [5]。

追加検証では1スレッドの低次元k-meansはsklearnより遅く、Digitsではsqrt版より通常Numba版が高速でした。常に同じ方式が最速とは限りません。再利用の大型計時は独立候補で実施し、a2 API統合後は小規模一致を確認。主表の12.90倍と掛け合わせません。T/Cの距離メモリは $O(N^2)$ 、SOMの64 MiB作業枠はRSS上限ではありません。BLASのスレッド制限はプロセス全体に作用します。

検証環境 Python 3.12.14 / NumPy 2.3.5 / SciPy 1.17.0 / scikit-learn 1.8.0 / Numba 0.67.0。一般公開・本番運用・他OSでの実測は未実施です。

今回の検証記録

- [1] restart_v2/evidence/kmeans_scratch_matched/{low_d,high_d,summary}.json
- [2] restart_v2/som_candidate/results/fullfit70k/{original,portable}_public_comparison.json
- [3] restart_v2/evidence/public_metrics/summary.json
- [4] restart_v2/README.md、evidence/ 内の配布QA記録、requirements-{foundation,tested}.txt
- [5] restart_v2/som_candidate/results/prepared_sweep70k/{fresh,prepared}.json
- [6] restart_v2/kmeans_candidate/results/matched_v2/{low_d,high_d,summary}.json
- [7] Numba公式 [Support Policy](#) / [Installation](#) 2026年9月30日確認

再現用のソースパッケージは別添です。入力データ本体・ビルド済みネイティブ・JITキャッシュは含めず、入力の由来・生成手順・ハッシュを残しています。これは新たな再構築版の成果で、旧版の同一バイト復旧ではありません。